

Thread 3 — Chromatogram Integration: allocation analysis

Why it is not a quick allocation win, unlike Quant & Output

2026-06-24 | branch `perf/runtime-profiling`

Bottom line. The Chromatogram Integration phase (~3.4 GB cumulative allocation, warm 2-file MTAC) is dominated by the *actual chromatogram signal* being materialized, plus one structure-of-arrays conversion copy. Unlike Quant & Output — whose 3.9 GB win came from skipping *redundant* string churn (99.5% duplicate accessions) — Integration’s allocation is mostly *necessary numeric data* that cannot be skipped. The single reducible item (the conversion copy) needs a moderate refactor, so it does *not* clear the “no excessive complexity” bar as a quick win. Recommendation: measure the copy precisely before deciding.

1 What the phase does

Three stages, in order:

1. `build_chromatograms` (`utils.jl:707`) — walks the scan range, runs the fused deconvolution kernel per scan, and **emits one** `MS2ChromObject` **per (precursor, scan) chromatogram point** into a `Vector{MS2ChromObject}` (pre-sized to `n_scans`×100). It returns `DataFrame{@view(chromatograms[utils.jl:878])}`.
2. `integrate_precursors` (`utils.jl:399`) — groups the chromatogram table by precursor and integrates each peak.
3. `integrate_chrom` (`integrate_chrom.jl`) — per precursor: Whittaker–Henderson smoothing → second-derivative bounds → baseline-subtracted trapezoidal area.

2 Where the 3.4 GB is — and is not

The integration loop is already allocation-lean. `integrate_precursors` (stages 2–3) was written to reuse memory:

- per-thread `WHWorkspace` and `Chromatogram` state are pre-allocated once (`utils.jl:431--432`) and reused across all precursors;
- every chromatogram slice is passed as an `@view` (`utils.jl:464--467`) — no per-precursor copies;
- results are written into caller-provided output vectors (`peak_area`, `new_best_scan`, `points_integrated`).

So the per-precursor smoothing/integration is *not* the allocator. (The line profiler agreed: `WHSmooth!` ≈0.11 GB, `build_chromatograms` leaf ≈0.15 GB — small.)

The allocation lives in stage 1, and it is mostly real data:

- **the `Vector{MS2ChromObject}` of chromatogram points** — this *is* the signal being integrated (every (precursor, scan) intensity in each precursor’s RT window). It scales with the data, not with waste.
- **the `AoS→DataFrame` conversion copy** (`DataFrame(@view(...))`, `utils.jl:878`) — the array-of-structs `MS2ChromObject` vector is copied field-by-field into the `DataFrame`’s columns; the original vector is then discarded. This is a transient $\sim 1\times$ duplicate of the chromatogram data, analogous to the Main Search `df_materialize` (0.78 GB there).

3 Why this is unlike the Quant & Output wins

	Quant & Output (Thread 2)	Chrom Integration (Thread 3)
What allocates	per-row <code>split</code> + <code>String</code> interning	chromatogram signal + <code>AoS→DF</code> copy
Nature	redundant (99.5% duplicate rows)	necessary (numeric data)
Fix	memoize & skip repeats	emit columns (SoA) instead of copy
Complexity	low (a <code>Set</code> /cache)	moderate (restructure emit path)
Result	−92% / −55%, bit-identical	only the copy is reducible

Thread 2 worked because the allocation was *repeated work over duplicate strings* — skippable with a tiny memo. Thread 3 has no such redundancy: the chromatogram points are distinct measured intensities. The only thing we are paying *twice* for is the `AoS` vector plus its `DataFrame` copy.

4 The one available lever, and the recommendation

Avoiding the conversion copy means having `build_chromatograms` write directly into column vectors (`rt`, `scan_idx`, `intensity`, `precursor_fraction_transmitted`, ...) — a structure-of-arrays emit path — and constructing the `DataFrame` from those columns with no per-element copy. That removes the duplicate but is a **moderate refactor** of the emit hot loop (and must stay bit-identical and not regress speed).

Recommendation. Before refactoring, *measure*: add `gc.bytes` sub-attribution (same method as the Main Search `MSLOOP` pass) to split the 3.4 GB into (a) the chromatogram `Vector`, (b) the `DataFrame` conversion copy, and (c) everything else. Refactor *only* if the copy is a clean $\gtrsim 1$ GB and the SoA emit stays simple. Otherwise, conclude Thread 3: the phase is near its necessary-data floor and further cuts would trade real complexity for little memory.

5 Sub-attribution results (measured) — the hypothesis was wrong

`gc.bytes` bucketing (env `PIONEER_DIAG_CHROM`, single-thread, 1-file MTAC, 2,726,965 chromatogram rows):

Step	GB	Note
integrate_precursors	0.724	largest — inside <code>integrate_chrom</code> (per precursor)
extract_chromatograms (Vector build)	0.592	per-scan deconv + the <code>MS2ChromObject</code> signal
sort_chromatograms_for_integration!	0.189	sortperm
df_convert (AoS→DataFrame copy)	0.074	the hypothesized target — negligible
get_isotopes_captured!	0.018	small

Corrected conclusion. The AoS→DataFrame copy is only **74 MB** — the measure-first guard worked, the SoA refactor is *not* worth it. The real targets are `integrate_chrom` (0.72 GB) and the `extract` Vector build (0.59 GB).

6 `integrate_chrom` drill-down — real but unattributed

`integrate_chrom` internal `gc_bytes` buckets: `WHSmooth!` 0.277, `subtractBaseline!` 0.149, `getIntegrationBounds!` 0.129, `fillState!+trapezoid` 0.099 GB — **distributed**, no single hotspot. A direct `@allocated` micro-benchmark confirms **~34 KB per call** (real, substantial). But the mechanism resisted every attribution:

- **Not type instability** — JET `@report_opt` on the real specialization: **0 reports**; typing `WHSmooth!`’s abstract `::AbstractVector` args to `::AbstractVector{Float32}` changed the allocation by *exactly zero* bytes.
- **Not captured-variable boxing** — `@code_warntype`: **0 Core.Box**.
- **Not explicit temporaries** — the WH solve chain (`whitsmddw!`, `build.DtD.bands!`) is fully in-place on the preallocated `WHWorkspace`; body `grep` finds no `zeros/similar/broadcast` (only two empty `Float32[]` debug stubs).

The allocation profiler attributes it to plain in-place loop lines (e.g. `x[i] /= rt_width`) that cannot self-allocate — consistent with a per-call cost of the **9 nested helper functions** (closure objects storing captured immutable refs do *not* show as `Core.Box`) that none of the surface diagnostics expose.

Recommendation. This is *not* a quick win and exceeds the “no excessive complexity” bar. Confirming and fixing it means **hoisting the 9 nested helpers to module scope** (the proven `_run_subset!` pattern) and re-measuring — a dedicated, separately bit-identical-verified refactor, worth ~1.3 GB phase + likely a speedup if it lands. Until then, Thread 3 is concluded: the integration phase is near its necessary-data floor on every *attributable* allocator.